Chronos

Low-Latency C++ Project Specification: Crypto Feed Handler (L2/L3)

Project Value Proposition

This multi-phase side project is meticulously designed to bridge existing C++ multithreading expertise with the deep, measurable low-latency systems programming skills demanded by elite financial technology firms and quantitative hedge funds. By implementing a full-stack, end-to-end data pipeline—from raw socket I/O and custom memory allocation to lock-free concurrent data structures and cache-aware algorithm design—I will gain hands-on experience in the critical path performance optimization techniques used in high-frequency trading (HFT). This initiative serves as a rigorous, long-term educational vehicle, culminating in a robust portfolio artifact that showcases deep, measurable expertise in systems-level C++, networking, and concurrency, significantly enhancing marketability for Quantitative Developer or Low-Latency Engineer roles.

Phase 1: Core Feed Handler (Single-Threaded & Low-Level Base)

Goal: Establish a fast, single-threaded connection to a cryptocurrency exchange data feed and parse incoming messages into highly efficient C++ objects.

Component	Objective	C++ Low-Level Focus
Socket I/O	Connect to a public exchange WebSocket API (e.g., Binance, Coinbase Pro) and efficiently read raw data.	Utilize **Boost.Asio** or pure **Linux Socket Programming (C API/epoll)** for high-performance non-blocking I/O. Focus on minimizing system calls.
Data Parser	Deserialize raw JSON/binary market data into an internal C++ structure.	**Manual memory parsing** (avoiding costly library overheads), using **std::span** for zero-copy views, and **placement new** for fast object creation within pre-allocated memory.
Data Structures	Design the **Market Data Event** structure for raw ticker/update	**Data-Oriented Design (DOD)**: Structs optimized for cache line

Component	Objective	C++ Low-Level Focus
	data.	usage and minimal padding. Use of **[[alignas(64)]]** to ensure cache line alignment where appropriate.
Logging	Implement a simple, synchronous, fire-and-forget logging function for system events.	Use of a pre-allocated **char buffer** or a simple buffered file write to reduce the number of system call overheads on the critical path.
Tooling	Setup the build environment and compiler settings.	Configure **CMake** with all relevant **compiler optimization flags** (-O3, -march=native, -DNDEBUG, etc.) for maximum speed.

Phase 2: Concurrent Processing Pipeline (Multithreading & Lock-Free)

Goal: Decouple network I/O from processing logic using high-throughput, deterministic inter-thread communication (ITC).

Component	Objective	C++ Low-Level Focus
Asynchronous Pipeline	Split the work across dedicated threads (I/O Thread \rightarrow Processing Thread).	**Thread Pinning (pthread_setaffinity_np)** to dedicate threads to specific CPU cores and eliminate latency from OS-level context switching.
Inter-Thread Communication (ITC)	Create the communication channel between the I/O thread and the processing thread.	Implement a **Single-Producer, Single-Consumer (SPSC) Lock-Free Queue (Ring Buffer)** using **std::atomic** and strict control over **memory ordering (std::memory_order_...)** to prevent CPU reordering issues.
Memory Management	Remove the dependency on the system heap (malloc/free) in the critical path.	Implement a **Custom Fixed-Size Memory Pool Allocator** (Object Pool) using a **singly-linked list of free blocks** to manage pre-allocated memory for Market Data Events.
Performance Measurement	Accurately measure the latency of data moving through the queue.	Use of the **TSC (Time-Stamp Counter) on x86** or **High-Resolution Clocks (std::chrono::high_resolution**

Component	Objective	C++ Low-Level Focus
		_clock) to measure and stamp latency in nanoseconds.

Phase 3: Order Book State & Arbitrage Logic (Complex State Management)

Goal: Maintain the high-speed market state and implement the core arbitrage detection logic with zero-allocation guarantee.

Component	Objective	C++ Low-Level Focus
Order Book	Maintain the current Level 2 (or Level 3) state for Bids and Offers.	Design the Order Book using highly efficient containers, possibly a pair of **cache-friendly std::vectors or std::arrays** for the Best Bids/Offers, ensuring quick lookups and updates.
Arbitrage Strategy	Implement a simple, high-speed logic to detect cross-exchange arbitrage opportunities based on Order Book data.	**Zero-Allocation Logic:** Ensure the entire arbitrage check and calculation runs without *any* heap allocation to guarantee consistent latency. Leverage **constexpr** where possible for compile-time calculation.
System Abstraction	Abstract the Order Book updates to allow for strategy extension.	Use of **CRTP (Curiously Recurring Template Pattern)** or **Policy-based Design** to achieve compile-time polymorphism, thereby avoiding the overhead of virtual function calls.
Order Gateway	Simulate the component responsible for sending trade signals out of the system.	Use an **SPSC queue** or a **pipe** to send minimal signals to a separate I/O thread, ensuring the main strategy thread is never blocked.

Phase 4: Reliability, Monitoring, and Optimisation (Production Readiness)

Goal: Introduce necessary infrastructure components to make the system robust, measurable, and ready for continuous performance tuning.

Component	Objective	C++ Low-Level Focus
Logging II (Async)	Upgrade the logging system for high-throughput, non-blocking logs.	Design a **Multi-Producer, Single-Consumer (MPSC) Queue** for log messages, allowing all threads to log

Component	Objective	C++ Low-Level Focus
		concurrently without blocking the critical data path.
Monitoring	Capture and analyze latency and throughput statistics in real-time.	Implementation of a **Custom High-Resolution Latency Histogram** to track p50, p90, p99, and p99.9 latencies directly in C++.
State Snapshots	Implement a mechanism to safely save and restore the Order Book state (for fast restarts/recovery).	Use of **Memory-Mapped Files (mmap on Linux)** for fast, persistent state saving without requiring costly deep serialization/deserialization routines.
Micro-Optimisation	Conduct a final, targeted low-level code review and refactoring round.	Using system tools like **perf** on Linux to identify and eliminate CPU cache misses, branch mispredictions, and to reduce the performance impact of **false sharing** between threads.